

How to Limit the Number of Concurrent Tasks in C#

In this article, I am going to discuss **How to Limit the Number of Concurrent Tasks in C# using SemaphoreSlim** with Examples. Please read our previous article where we discussed [How to Execute Multiple Tasks using the WhenAll Method in C#](#) with Examples. At the end of this article, you will understand the following two pointers in depth.

1. [How to Limit the Number of Concurrent Tasks in C#?](#)
2. [How to Handle the Response of Multiple Tasks when executed using Task.WhenAll Method?](#)

How to Limit the Number of Concurrent Tasks in C#?

In the below example, we are processing 100000 tasks concurrently.

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            var stopwatch = new Stopwatch();
            stopwatch.Start();
            Console.WriteLine($"Main Thread Started");
            List<CreditCard> creditCards =
CreditCard.GenerateCreditCards(100000);
            Console.WriteLine($"Credit Card Generated :
{creditCards.Count}");

            ProcessCreditCards(creditCards);

            Console.WriteLine($"Main Thread Completed");
            stopwatch.Stop();
            Console.WriteLine($"Main Thread Execution Time
{stopwatch.ElapsedMilliseconds / 1000.0} Seconds");
            Console.ReadKey();
        }
        public static async void ProcessCreditCards(List<CreditCard>
creditCards)
        {
            var stopwatch = new Stopwatch();
            stopwatch.Start();
            var tasks = new List<Task<string>>();
            await Task.Run(() =>
            {
                foreach (var creditCard in creditCards)
```

```

        {
            var response = ProcessCard(creditCard);
            tasks.Add(response);
        }
    });

    //It will execute all the tasks concurrently
    await Task.WhenAll(tasks);
    stopwatch.Stop();
    Console.WriteLine($"Processing of {creditCards.Count} Credit
Cards Done in {stopwatch.ElapsedMilliseconds/1000.0} Seconds");
}

public static async Task<string> ProcessCard(CreditCard creditCard)
{
    await Task.Delay(1000);
    string message = $"Credit Card Number: {creditCard.CardNumber}
Name: {creditCard.Name} Processed";
    return message;
}
}
public class CreditCard
{
    public string CardNumber { get; set; }
    public string Name { get; set; }
    public static List<CreditCard> GenerateCreditCards(int number)
    {
        List<CreditCard> creditCards = new List<CreditCard>();
        for (int i = 0; i < number; i++)
        {
            CreditCard card = new CreditCard()
            {
                CardNumber = "10000000" + i,
                Name = "CreditCard-" + i
            };
            creditCards.Add(card);
        }
        return creditCards;
    }
}
}

```

Output:

```

Main Thread Started
Credit Card Generated : 100000
Main Thread Completed
Main Thread Execution Time 0.102 Seconds
Processing of 100000 Credit Cards Done in 9.919 Seconds

```

Here, we have processed 100000 tasks simultaneously. But there might be problems when we execute a huge number of tasks simultaneously, for example, the server might be unable to handle such a huge request, or if we send 100000 HTTP requests to a server, it might be blocked or down.

So, instead of sending 100000 HTTP requests at a time or processing 100000 tasks simultaneously, what we need to do is, we need to send them as a batch or process the tasks as a batch and we can do this in C# by using **SemaphoreSlim**. With **SemaphoreSlim**, we are able to limit the number of concurrent tasks that will be executed with the Task.WhenAll method. Let us understand this with an example.

Example to Understand How to Limit the Number of Concurrent Tasks in C# using SemaphoreSlim:

For a better understanding, we are not going to process 100000 Credit Cards. What we will do is we will process 15 credit cards with a batch of 3. That means five batches will be executed to process the 15 credit cards. Let us see how we can achieve this.

First, we need to create an instance of SemaphoreSlim class as follows. Here, we are passing the initial capacity as 3. That means at a time 3 threads are allowed to execute the tasks.

```
SemaphoreSlim semaphoreSlim = new SemaphoreSlim(3);
```

So, here, what SemaphoreSlim is doing is if we have more than 3 tasks been running, we are going to wait and we're going to wait until the semaphore gets released. If you are new to SemaphoreSlim, please read the following article where we discussed SemaphoreSlim in detail.

<https://dotnettutorials.net/lesson/semaphoreslim-class-in-csharp/>

Next, we need to convert the following piece of code of our ProcessCreditCards method to use SemaphoreSlim.

```
await Task.Run(() =>
{
    foreach (var creditCard in creditCards)
    {
        var response = ProcessCard(creditCard);
        tasks.Add(response);
    }
});
```

The following code shows how to use SemaphoreSlim to limit the number of concurrent tasks to be executed simultaneously. As we are using the WaitAsync method so, we need to use the async lambda expression, and also, we need to use await operator while calling the ProcessCard function. We need to release the semaphore inside the finally block which make sure that if an exception occurred then also the semaphoreslim object release the thread so that other task can be executed by the thread.

```

//Need to use async lambda expression
tasks = creditCards.Select(async card =>
{
    //This will tell if we have more than 4000 tasks are running,
    //we are going to wait and '
    //we're going to wait until the semaphore gets released.
    await semaphoreSlim.WaitAsync();

    //Need to use await operator here as we are using asynchronous WaitAsync
    try
    {
        return await ProcessCard(card);
    }
    finally
    {
        //Release the semaphore
        semaphoreSlim.Release();
    }
}).ToList();

```

Async Lambda Expression

Async Wait Method

Calling the ProcessCard Method using await operator

Release the Semaphore so that other tasks can be executed

Complete Example Code:

The following is the complete example code that shows how to use SemaphoreSlim to limit the number of concurrent tasks. Here, it will execute the tasks in batches, and in each batch, it will execute a maximum of three tasks. In the below example, we need to include the System.Threading and System.Linq namespaces. SemaphoreSlim class belongs to System.Threading namespace and as we use LINQ queries so we need to include System.Linq namespace.

```

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Threading;
using System.Threading.Tasks;
using System.Linq;
namespace AsynchronousProgramming
{
    class Program
    {
        //Allowing Maximum 3 tasks to be executed at a time
        static SemaphoreSlim semaphoreSlim = new SemaphoreSlim(3);
        static void Main(string[] args)
        {
            var stopwatch = new Stopwatch();
            Console.WriteLine($"Main Thread Started");
            //Generating 15 Credit Cards

```

```

        List<CreditCard> creditCards =
CreditCard.GenerateCreditCards(15);
        Console.WriteLine($"Credit Card Generated :
{creditCards.Count}");

        ProcessCreditCards(creditCards);

        Console.WriteLine($"Main Thread Completed");
        Console.ReadKey();
    }
    public static async void ProcessCreditCards(List<CreditCard>
creditCards)
    {
        var stopwatch = new Stopwatch();
        stopwatch.Start();
        var tasks = new List<Task<string>>();
        //Need to use async lambda expression
        tasks = creditCards.Select(async card =>
        {
            //This will tell if we have more than 4000 tasks are running,
            //we are going to wait and '
            //we're going to wait until the semaphore gets released.
            await semaphoreSlim.WaitAsync();
            //Need to use await operator here as we are using
asynchronous WaitAsync
            try
            {
                return await ProcessCard(card);
            }
            finally
            {
                //Release the semaphore
                semaphoreSlim.Release();
            }
        })
        }).ToList();

        //It will execute a maximum of 3 tasks at a time
        await Task.WhenAll(tasks);
        stopwatch.Stop();
        Console.WriteLine($"Processing of {creditCards.Count} Credit
Cards Done in {stopwatch.ElapsedMilliseconds/1000.0} Seconds");
    }

    public static async Task<string> ProcessCard(CreditCard creditCard)
    {
        await Task.Delay(1000);
        string message = $"Credit Card Number: {creditCard.CardNumber}
Name: {creditCard.Name} Processed";
        Console.WriteLine($"Credit Card Number: {creditCard.CardNumber}
Processed");
    }

```

```

        return message;
    }
}
public class CreditCard
{
    public string CardNumber { get; set; }
    public string Name { get; set; }
    public static List<CreditCard> GenerateCreditCards(int number)
    {
        List<CreditCard> creditCards = new List<CreditCard>();
        for (int i = 0; i < number; i++)
        {
            CreditCard card = new CreditCard()
            {
                CardNumber = "10000000" + i,
                Name = "CreditCard-" + i
            };
            creditCards.Add(card);
        }
        return creditCards;
    }
}
}

```

Output:

```

Main Thread Started
Credit Card Generated : 15
Main Thread Completed
Credit Card Number: 100000002 Processed
Credit Card Number: 100000001 Processed
Credit Card Number: 100000000 Processed
Credit Card Number: 100000005 Processed
Credit Card Number: 100000003 Processed
Credit Card Number: 100000004 Processed
Credit Card Number: 100000006 Processed
Credit Card Number: 100000008 Processed
Credit Card Number: 100000007 Processed
Credit Card Number: 100000009 Processed
Credit Card Number: 100000010 Processed
Credit Card Number: 100000011 Processed
Credit Card Number: 100000013 Processed
Credit Card Number: 100000012 Processed
Credit Card Number: 100000014 Processed
Processing of 15 Credit Cards Done in 5.112 Seconds

```

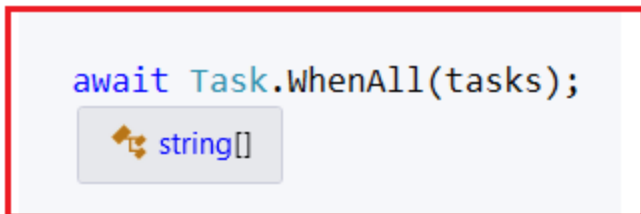
Here, please observe the output. It is taking a little more than 5 seconds and this is expected. Because it will execute all the tasks in five batches. And if you notice we have delayed the ProcessCard Execution by 1

second. That means executing one batch will take little more than 1 second and the same for all 5 batches, and hence the overall time is little more than 5 seconds.

How to Handle Response when Executing Multiple Tasks using Tasks.WhenAll Method in C#?

Now, let us understand how to handle the response when executing multiple tasks concurrently using Tasks.WhenAll Method in C#. We know Tasks.WhenAll tells that please wait for all of the tasks to be done before continuing with the execution of the rest part of the method. That means it will make sure once all the tasks are completed then only, we can proceed further to execute the rest part of the method.

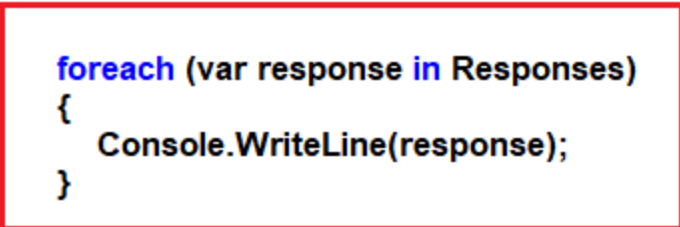
If you further notice the return type of the ProcessCard card method is Task<string>. That means the method is returning something. As the WhenAll method execute all the tasks that means all the tasks return some data. How we can fetch such that? Let us see. Please have a look at the below image. If you put the mouse pointer over the await operator, then you will see that it is returning a string array.



So, we can store the response in a string array as follows:

```
string[] Responses= await Task.WhenAll(tasks);
```

Then by using a foreach loop we can access the result of each task as follows.



Complete Example Code:

Whatever we discussed is shown in the below example.

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Threading;
using System.Threading.Tasks;
using System.Linq;
namespace AsynchronousProgramming
{
```

```

class Program
{
    //Allowing Maximum 3 tasks to be executed at a time
    static SemaphoreSlim semaphoreSlim = new SemaphoreSlim(3);
    static void Main(string[] args)
    {
        var stopwatch = new Stopwatch();
        Console.WriteLine($"Main Thread Started");
        //Generating 15 Credit Cards
        List<CreditCard> creditCards =
CreditCard.GenerateCreditCards(15);
        Console.WriteLine($"Credit Card Generated :
{creditCards.Count}");
        ProcessCreditCards(creditCards);
        Console.WriteLine($"Main Thread Completed");
        Console.ReadKey();
    }
    public static async void ProcessCreditCards(List<CreditCard>
creditCards)
    {
        var stopwatch = new Stopwatch();
        stopwatch.Start();
        var tasks = new List<Task<string>>();
        //Need to use async lambda expression
        tasks = creditCards.Select(async card =>
        {
            await semaphoreSlim.WaitAsync();
            try
            {
                return await ProcessCard(card);
            }
            finally
            {
                semaphoreSlim.Release();
            }
        }).ToList();
        string[] Responses = await Task.WhenAll(tasks);
        //var Responses = await Task.WhenAll(tasks);
        foreach (var response in Responses)
        {
            Console.WriteLine(response);
        }
        stopwatch.Stop();
        Console.WriteLine($"Processing of {creditCards.Count} Credit
Cards Done in {stopwatch.ElapsedMilliseconds / 1000.0} Seconds");
    }
    public static async Task<string> ProcessCard(CreditCard creditCard)
    {
        await Task.Delay(1000);
        string message = $"Credit Card Number: {creditCard.CardNumber}
Name: {creditCard.Name} Processed";
    }
}

```

```

        //Console.WriteLine($"Credit Card Number: {creditCard.CardNumber}
Processed");
        return message;
    }
}
public class CreditCard
{
    public string CardNumber { get; set; }
    public string Name { get; set; }
    public static List<CreditCard> GenerateCreditCards(int number)
    {
        List<CreditCard> creditCards = new List<CreditCard>();
        for (int i = 0; i < number; i++)
        {
            CreditCard card = new CreditCard()
            {
                CardNumber = "10000000" + i,
                Name = "CreditCard-" + i
            };
            creditCards.Add(card);
        }
        return creditCards;
    }
}
}

```

Output:

```

Main Thread Started
Credit Card Generated : 15
Main Thread Completed
Credit Card Number: 100000000 Name: CreditCard-0 Processed
Credit Card Number: 100000001 Name: CreditCard-1 Processed
Credit Card Number: 100000002 Name: CreditCard-2 Processed
Credit Card Number: 100000003 Name: CreditCard-3 Processed
Credit Card Number: 100000004 Name: CreditCard-4 Processed
Credit Card Number: 100000005 Name: CreditCard-5 Processed
Credit Card Number: 100000006 Name: CreditCard-6 Processed
Credit Card Number: 100000007 Name: CreditCard-7 Processed
Credit Card Number: 100000008 Name: CreditCard-8 Processed
Credit Card Number: 100000009 Name: CreditCard-9 Processed
Credit Card Number: 1000000010 Name: CreditCard-10 Processed
Credit Card Number: 1000000011 Name: CreditCard-11 Processed
Credit Card Number: 1000000012 Name: CreditCard-12 Processed
Credit Card Number: 1000000013 Name: CreditCard-13 Processed
Credit Card Number: 1000000014 Name: CreditCard-14 Processed
Processing of 15 Credit Cards Done in 5.097 Seconds

```

Real-Time Example:

In the real-time application when we are calling an API, we will not get any string data. In most cases, we will get JSON data. If we are getting some JSON data then how we will handle it? We need to use JSON Serializer. To use JSON serializer, first, we need to include Newtonsoft.Json DLL from the NuGet. Once you install Newtonsoft.Json DLL from the NuGet, then modify the code as follows which shows how to use JSON serializer. The following example code is self-explained, so please go through the comment lines.

```
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Threading;
using System.Threading.Tasks;
using System.Linq;
using Newtonsoft.Json;
namespace AsynchronousProgramming
{
    class Program
    {
        //Allowing Maximum 3 tasks to be executed at a time
        static SemaphoreSlim semaphoreSlim = new SemaphoreSlim(3);
        static void Main(string[] args)
        {
            var stopwatch = new Stopwatch();
            Console.WriteLine($"Main Thread Started");
            //Generating 15 Credit Cards
            List<CreditCard> creditCards =
CreditCard.GenerateCreditCards(15);
            Console.WriteLine($"Credit Card Generated :
{creditCards.Count}");
            ProcessCreditCards(creditCards);
            Console.WriteLine($"Main Thread Completed");
            Console.ReadKey();
        }
        public static async void ProcessCreditCards(List<CreditCard>
creditCards)
        {
            var stopwatch = new Stopwatch();
            stopwatch.Start();
            var tasks = new List<Task<string>>();
            //Need to use async lambda expression
            tasks = creditCards.Select(async card =>
            {
                await semaphoreSlim.WaitAsync();
                try
                {
                    return await ProcessCard(card);
                }
                finally
                {

```

```

        semaphoreSlim.Release();
    }
}).ToList();
//Return the response a string array
var Responses = await Task.WhenAll(tasks);
//Creating a collection to hold the responses
List<CreditCardResponse> creditCardResponses = new
List<CreditCardResponse>();
//Looping through the string array
foreach (var response in Responses)
{
    //Here, the string is a JSON string
    //Converting the JSON String to .NET Object
    (CreditCardResponse) using
    //JsonConvert class DeserializeObject
    CreditCardResponse creditCardResponse =
JsonConvert.DeserializeObject<CreditCardResponse>(response);
    //Adding the .NET Object into the response collection
    creditCardResponses.Add(creditCardResponse);
}
//Printing all the approved credit cards using a foreach loop
Console.WriteLine("\nApproved Credit Cards");
foreach(var item in creditCardResponses.Where(card =>
card.IsProcessed == true))
{
    Console.WriteLine($"Card Number: {item.CardNumber}, Name:
{item.Name}");
}
//Printing all the rejected credit cards using a foreach loop
Console.WriteLine("\nRejected Credit Cards");
foreach (var item in creditCardResponses.Where(card =>
card.IsProcessed == false))
{
    Console.WriteLine($"Card Number: {item.CardNumber}, Name:
{item.Name}");
}
}
public static async Task<string> ProcessCard(CreditCard creditCard)
{
    await Task.Delay(1000);

    var creditCardResponse = new CreditCardResponse
    {
        CardNumber = creditCard.CardNumber,
        Name = creditCard.Name,
        //Logic to Decide whether the card is processed or rejected
        //If modulus 2 is 0, the processed else rejected
        IsProcessed = creditCard.CardNumber % 2 == 0 ? true : false
    };
    //Converting the .NET Object to JSON string

```

```

        string jsonString =
JsonConvert.SerializeObject(creditCardResponse);
        //Return the JSON String
        return jsonString;
    }
}
public class CreditCard
{
    public long CardNumber { get; set; }
    public string Name { get; set; }
    public static List<CreditCard> GenerateCreditCards(int number)
    {
        List<CreditCard> creditCards = new List<CreditCard>();
        for (int i = 0; i < number; i++)
        {
            CreditCard card = new CreditCard()
            {
                CardNumber = 100000000 + i,
                Name = "CreditCard-" + i
            };
            creditCards.Add(card);
        }
        return creditCards;
    }
}
//This class will hold the response after processing the Credit card
public class CreditCardResponse
{
    public long CardNumber { get; set; }
    public string Name { get; set; }
    public bool IsProcessed { get; set; }
}
}

```

Output:

```
Main Thread Started
Credit Card Generated : 15
Main Thread Completed

Approved Credit Cards
Card Number: 10000000, Name: CreditCard-0
Card Number: 10000002, Name: CreditCard-2
Card Number: 10000004, Name: CreditCard-4
Card Number: 10000006, Name: CreditCard-6
Card Number: 10000008, Name: CreditCard-8
Card Number: 10000010, Name: CreditCard-10
Card Number: 10000012, Name: CreditCard-12
Card Number: 10000014, Name: CreditCard-14

Rejected Credit Cards
Card Number: 10000001, Name: CreditCard-1
Card Number: 10000003, Name: CreditCard-3
Card Number: 10000005, Name: CreditCard-5
Card Number: 10000007, Name: CreditCard-7
Card Number: 10000009, Name: CreditCard-9
Card Number: 10000011, Name: CreditCard-11
Card Number: 10000013, Name: CreditCard-13
```

In the next article, I am going to discuss [How to Cancel a Long-Running Task using Cancellation Token in C#](#) with Examples. Here, in this article, I try to explain **How to Limit the Number of Concurrent Tasks in C# using SemaphoreSlim** with Examples. I hope you enjoy this **How to Limit the Number of Concurrent Tasks in C# using SemaphoreSlim with Examples** article.